

Unified Deployment Guide

Gema Event Management Platform
Generated: 2026-02-25

Table of Contents

- [Overview](#)
- [Prerequisites](#)
- [Pre-Deployment Checklist](#)
- [Deployment Environments](#)
 - [Hostinger KVM1 VPS \(Recommended\)](#)
 - [Render Deployment](#)
 - [AWS EC2 Deployment](#)
 - [Docker Deployment](#)
- [Frontend Deployment Options](#)
- [Database & Services Configuration](#)
- [Post-Deployment Tasks](#)
- [Rollback & Troubleshooting](#)

1. Overview

The GEMA platform uses a modern decoupled architecture:

- **Frontend:** React 18 + Vite (Static Build)
- **Backend:** Node.js 20.x + Express + TypeScript
- **Database:** MongoDB Atlas
- **Services:** Firebase (Auth/Notifications), Stripe (Payments), Cloudinary (Media)

2. Prerequisites

Before deploying, ensure you have credentials for:

- MongoDB Atlas cluster
- Firebase project (Service Account JSON)
- Stripe account (Secret, Publishable, and Webhook keys)
- Cloudinary account
- Domain name with DNS access

3. Pre-Deployment Checklist

- [] All tests passing locally.
- [] TypeScript builds succeed (`npm run build`).
- [] `SLUG_MIGRATION` completed if bridging legacy IDs to Slugs.
- [] Environment Variables (`.env` for backend, `.env.production` for frontend) prepared.
- [] Database backed up (if updating existing production).

4. Deployment Environments

Hostinger KVM1 VPS (Recommended)

Suitable for staging, MVP launches, and low-to-moderate traffic (<1000 concurrent).

1. **Server Specs:** 4GB RAM, 1 vCPU, Ubuntu 22.04 LTS.
2. **Automated Setup:**

```
ssh root@YOUR_SERVER_IP
wget https://raw.githubusercontent.com/EshaanManchanda/GEMA-Project/backend_auth/setup-server.sh
chmod +x setup-server.sh
sudo ./setup-server.sh
```

3. **Environment:** Configure `.env` in `/var/www/gema/backend/` and `/var/www/gema/frontend/`.

4. **Deploy Script:** `./deploy.sh` in `/var/www/gema`.
5. **SSL:** `certbot --nginx -d your-domain.com`.

Render Deployment (Cloud PaaS)

1. **Connect Repo:** In Render Dashboard, create new Web Service.
2. **Settings:**
 - Environment: Node
 - Build Command: `npm run build`
 - Start Command: `npm start`
3. **Environment Variables:** Add all `.env` variables via Render UI.

AWS EC2 Deployment

1. **Instance:** t3.medium Ubuntu 22.04.
2. **Setup:** Install Node.js 18+, PM2, and Nginx.
3. **Run:**

```
npm run build
pm2 start ecosystem.config.js --env production
```

4. **Proxy:** Configure Nginx as a reverse proxy for `localhost:5000`.

Docker Deployment

1. Build via `docker-compose up -d`.
2. Includes mongo image for local DB or connects to Atlas via env variables.
3. Requires `NODE_ENV=production`.

5. Frontend Deployment Options

- **Vite Static Build:** Run `npm run build` in the `frontend/` folder.
- **Nginx (Self-Hosted):** Serve the `dist/` directory via Nginx block.
- **Vercel / Netlify:**
 - Connect Git Repo.
 - Build command: `npm run build`.
 - Output directory: `dist`.

6. Database & Services Configuration

- **MongoDB Atlas:** Whitelist your server IP (`0.0.0.0/0` for Render/Vercel).
- **Stripe Webhooks:** Point to `https://your-domain.com/api/payments/webhook`. Wait for `payment_intent.succeeded` events.
- **Cloudinary:** Ensure `CLOUDINARY_SECURE=true` is set.

7. Post-Deployment Tasks

- **Health Check:** `curl https://your-domain.com/api/health`
- **Verify DB Connection:** Ensure "MongoDB connected successfully" is in PM2 logs.
- **Test Frontend Flow:** Registration, Event Booking.

8. Rollback & Troubleshooting

- **PM2 Logs:** `pm2 logs gema-backend`
- **Nginx Logs:** `tail -f /var/log/nginx/error.log`
- **Rollback Script (Hostinger):** `./deploy.sh --rollback`
- **Frontend Vercel Rollback:** Use Vercel Dashboard to instantly revert to previous deployments.